

Student Data Integration Project

Integrating Student Details, Academic Performance, Skills, and Placement Records using Pandas

Edrin J S

192524041

1. Introduction

This report documents the process of integrating four datasets supplied for this assignment - Student Details, Academic Performance, Skills Assessment, and Placement Information - into a single, analysis-ready table using Python and Pandas. The work was carried out in a Jupyter notebook, and this report summarises the approach, the data-quality issues identified, the cleaning decisions made, the join strategy selected, and the validation performed before the final dataset was exported.

Student Details, the master roster, contained 1,004 rows (1,000 unique students once exact duplicates were removed). Academic Performance contained 963 rows, Skills Assessment 870 rows, and Placement Information 661 rows. The difference in row counts across the four files is expected, since not every registered student has an academic record, has completed the skills assessment, or has been interviewed by a company.

2. Loading and Inspecting the Data

Each file was loaded independently with `pd.read_csv()` and inspected before any changes were made, so that the structure, column names, and data types of each dataset were fully understood beforehand. Figure 1 shows the four datasets being loaded and their shapes confirmed.

```
import pandas as pd
import numpy as np

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 140)

BASE_DIR = r"C:\query processing"

student_raw = pd.read_csv(fr"{BASE_DIR}\Student details.csv")
academic_raw = pd.read_csv(fr"{BASE_DIR}\Academic_Performance.csv")
skills_raw = pd.read_csv(fr"{BASE_DIR}\Skills_Assessment.csv")
placement_raw = pd.read_csv(fr"{BASE_DIR}\Placement_Information.csv")

for name, df in [("Student_Details", student_raw), ("Academic_Performance", academic_raw),
                 ("Skills_Assessment", skills_raw), ("Placement_Information", placement_raw)]:
    print(f"{name:22s} shape={df.shape}")
✓ [1] 495ms

Student_Details      shape=(1004, 6)
Academic_Performance shape=(963, 4)
Skills_Assessment    shape=(870, 4)
Placement_Information shape=(661, 4)
```

Figure 1. Loading the four source files and confirming their shapes.

An `inspect()` function was then used to review the columns, data types, and sample records of each dataset in turn. This step was essential in surfacing the first data-quality issue: the student identifier column was named differently across files (`Student_ID`, `StudentID`, and `'student id'`). Figure 2 shows this inspection being carried out on the `Student Details` file.

```
def inspect(name, df):
    print(f"\n{'='*70}\n{name}\n{'='*70}")
    print("Shape:", df.shape)
    print("\nColumns & dtypes:\n", df.dtypes)
    display(df.head(3))
    for c in df.select_dtypes(include='object').columns:
        print(f"  {c}: {df[c].nunique()} unique -> {df[c].dropna().unique()[:8]}")

inspect("Student_Details", student_raw)
inspect("Academic_Performance", academic_raw)
inspect("Skills_Assessment", skills_raw)
inspect("Placement_Information", placement_raw)
✓ [2] 109ms
```

```
=====
Student_Details
=====
Shape: (1004, 6)

Columns & dtypes:
Student_ID      str
Full Name       str
Gender          str
Department      str
Admission Year  int64
```

	Student_ID	Full Name	Gender	Department	Admission_Year	Email
0	STU0001	Rohan Menon	Male	Information Technology	2021	NaN
1	STU0002	Aman Rao	Female	Civil	2022	aman.rao1@college.edu
2	STU0003	Isha Agarwal	Male	Computer Science	2022	isha.agarwal2@college.edu

Figure 2. Inspecting column names, data types and sample rows in `Student Details`.

3. Data Quality Assessment

A `quality_report()` function was written to check each dataset systematically for missing values, fully duplicated rows, and duplicate keys. Figure 3 shows this check being run against all four datasets, including the result for `Placement Information`, which returned 231 duplicate `Student_ID` values.

```
def quality_report(name, df, key_col):
    print(f"\n{' '*70}\n{name} (key={key_col})\n{' '*70}")
    print("Missing values:\n", df.isna().sum()[df.isna().sum() > 0])
    print("Fully duplicated rows:", df.duplicated().sum())
    print("Duplicate keys:", df[key_col].duplicated().sum())

quality_report("Student_Details", student_raw, "Student_ID")
quality_report("Academic_Performance", academic_raw, "StudentID")
quality_report("Skills_Assessment", skills_raw, "student id")
quality_report("Placement_Information", placement_raw, "Student_ID")
✓ [3] 37ms
dtype: int64
Fully duplicated rows: 0
Duplicate keys: 0

=====
Placement_Information (key=Student_ID)
=====
Missing values:
Package_LPA    148
dtype: int64
Fully duplicated rows: 6
Duplicate keys: 231
```

Figure 3. Systematic missing-value, duplicate-row and duplicate-key check across all four datasets.

The 231 duplicate keys in Placement Information were expected and are not treated as an error: a student who interviewed with more than one company legitimately appears more than once in that file. This distinction - a duplicate row versus a duplicate key that reflects a genuine one-to-many relationship - is addressed further in Section 6.

In summary, the following issues were identified before any cleaning was performed:

- Missing values in Email, CGPA, Soft_Skill_Score, and Package_LPA.
- CGPA stored as text rather than a numeric type, with some entries recorded as the literal string "NA" or left blank.
- Inconsistent spelling of categorical values, e.g. Gender (Male / M / male) and Placement_Status (Placed / placed / Selected).
- Different column names for the same underlying student identifier across all four files.
- 4 exact duplicate rows in Student Details and 6 in Placement Information.
- 3 students in Academic Performance recorded twice with conflicting CGPA values - a duplicate-key issue rather than a duplicate row.

4. Standardization and Cleaning

All cleaning was performed on in-memory copies of the dataframes; the original CSV files were not modified at any point. The identifier columns were renamed to a single common name, Student_ID, and standardized to a consistent uppercase, whitespace-stripped format so that the same student would match correctly across files. Exact duplicate rows were then dropped, the duplicate-key conflicts in Academic Performance were resolved by retaining the later record for each affected student, CGPA was converted to a numeric type, and the Gender and Placement_Status categories were standardized. Figure 4 shows this full standardization step, including the three duplicate-key student IDs identified in Academic Performance (STU0130, STU0394, STU0664).

```

academic = academic_raw.copy()
skills = skills_raw.copy()
placement = placement_raw.copy()

student = student.rename(columns={"Full Name": "Full_Name"})
academic = academic.rename(columns={"StudentID": "Student_ID"})
skills = skills.rename(columns={"student id": "Student_ID"})

for df in (student, academic, skills, placement):
    df["Student_ID"] = df["Student_ID"].astype(str).str.strip().str.upper()

student = student.drop_duplicates()
placement = placement.drop_duplicates()

dup_ids = academic.loc[academic['Student_ID'].duplicated(keep=False), 'Student_ID'].unique()
print("Duplicate-key students in Academic:", list(dup_ids))
academic = academic.sort_values('Student_ID').drop_duplicates(subset='Student_ID', keep='last')

academic['CGPA'] = academic['CGPA'].replace({'NA': np.nan, '' : np.nan}).astype(float)

student['Gender'] = student['Gender'].str.strip().str.upper().map(
    {'MALE': 'Male', 'M': 'Male', 'FEMALE': 'Female', 'F': 'Female'})

skills['Primary_Language'] = skills['Primary_Language'].str.strip().str.title().replace({'SQL': 'SQL'})

status_map = {'Placed': 'Placed', 'placed': 'Placed', 'Selected': 'Placed',
              'Not Placed': 'Not Placed', 'Rejected': 'Not Placed'}
placement['Placement_Status'] = placement['Placement_Status'].str.strip().map(status_map)

print("Standardization complete.")
✓ [4] 40ms

Duplicate-key students in Academic: ['STU0130', 'STU0394', 'STU0664']
Standardization complete.

```

Figure 4. Standardizing identifiers, removing duplicates, and resolving conflicting academic records.

5. Checking for Unmatched Records

With identifiers standardized, a set comparison between the Student_ID values in Student Details and each of the other three files was used to identify unmatched records directly. This produced the figures shown in Figure 5: 40 registered students have no academic record, 130 have no skills-assessment record, and 570 have never been interviewed.

```

all_students = set(student['Student_ID'])
missing_academic = sorted(all_students - set(academic['Student_ID']))
missing_skills = sorted(all_students - set(skills['Student_ID']))
not_interviewed = sorted(all_students - set(placement['Student_ID']))

print("Registered students:", len(all_students))
print("Missing academic record:", len(missing_academic))
print("Missing skills record:", len(missing_skills))
print("Never interviewed:", len(not_interviewed))
✓ [5] 10ms

Registered students: 1000
Missing academic record: 40
Missing skills record: 130
Never interviewed: 570

```

Figure 5. Identifying unmatched records between Student Details and the other three datasets.

6. Join Strategy and Integration

The assignment requirement was that every registered student remain present in the final dataset, regardless of whether their academic, skills, or placement information was complete. This requirement determines the join

type directly: a LEFT JOIN was used throughout, with Student Details as the base (left) table, so that Academic Performance, Skills Assessment, and Placement Information were each merged onto it without removing any student who lacked a matching record.

An inner join was deliberately avoided, since it would have silently dropped the 40 students with no academic record and the 570 students who never interviewed - all of whom are still legitimately registered students whose absence from certain files is informative, not an error to be discarded.

Placement Information required additional care because of its one-to-many relationship with Student Details: a student who received multiple interview opportunities appears once per interview. Merging this table directly onto the one-row-per-student base would have caused every other column for that student (name, department, academic record, and so on) to be duplicated once per interview - reproducing the exact 'duplicate student' problem the join strategy was designed to avoid. To prevent this, Placement Information was first aggregated with `groupby('Student_ID')` into one row per student, producing derived fields for the number of interviews, number of offers, placement status, and average and best package received, before being merged onto the main table. This preserves every interview record (the cleaned interview-level table remains available separately) while keeping the final dataset at the correct one-row-per-student grain.

Figure 6 shows the three merge operations, the aggregation of Placement Information, and the row-count assertions used to confirm the row count remained at 1,000 throughout - producing a final shape of 1,000 rows by 17 columns.

```
merged = student.merge(academic, on='Student_ID', how='left', validate='one_to_one')
assert len(merged) == len(student)

merged = merged.merge(skills, on='Student_ID', how='left', validate='one_to_one')
assert len(merged) == len(student)

placement_agg = placement.groupby('Student_ID').agg(
    Num_Interviews = ('Company_Name', 'count'),
    Num_Offers      = ('Placement_Status', lambda s: (s == 'Placed').sum()),
    Best_Package_LPA = ('Package_LPA', 'max'),
    Avg_Package_LPA  = ('Package_LPA', 'mean'),
).reset_index()
placement_agg['Is_Placed'] = placement_agg['Num_Offers'] > 0
placement_agg[['Best_Package_LPA', 'Avg_Package_LPA']] = placement_agg[['Best_Package_LPA', 'Avg_Package_LPA']].round(2)

final_df = merged.merge(placement_agg, on='Student_ID', how='left', validate='one_to_one')
final_df['Num_Interviews'] = final_df['Num_Interviews'].fillna(0).astype(int)
final_df['Num_Offers']     = final_df['Num_Offers'].fillna(0).astype(int)
final_df['Is_Placed']     = final_df['Is_Placed'].fillna(False)

assert len(final_df) == len(student)
print("Final shape:", final_df.shape)
✓ [6] 163ms

Final shape: (1000, 17)
```

Figure 6. Left-join integration of Academic Performance and Skills, and aggregation and integration of Placement Information.

7. Post-Merge Validation

After the final merge, the integrated dataset was validated against the same criteria used to assess the source files: duplicate keys, duplicate rows, missing values by column, and the number of unique students represented. Figure 7 shows this validation, together with the export of the final file to Final_Integrated_Dataset.csv.

```
print("Duplicate keys:", final_df['Student_ID'].duplicated().sum())
print("Duplicate rows:", final_df.duplicated().sum())
print("Missing values:\n", final_df.isna().sum())
print("Unique students:", final_df['Student_ID'].nunique(), "==" rows:", len(final_df))
✓ [7] 13ms
Backlogs          40
Attendance_%      40
Primary_Language  130
Certifications    130
Soft_Skill_Score  140
Num_Interviews    0
Num_Offers        0
Best_Package_LPA  638
Avg_Package_LPA   638
Is_Placed         0
dtype: int64
Unique students: 1000 == rows: 1000

OUT_PATH = fr"{BASE_DIR}\Final_Integrated_Dataset.csv"
final_df.to_csv(OUT_PATH, index=False)
print("Saved:", OUT_PATH)
✓ [8] 33ms

Saved: C:\query processing\Final_Integrated_Dataset.csv
```

Figure 7. Final validation checks and export of the integrated dataset.

The result confirmed 1,000 unique Student_ID values matching exactly 1,000 rows, zero duplicate keys, and zero duplicate rows. The remaining missing values (40 in Backlogs and Attendance_%, 130 in Primary_Language and Certifications, 638 in Best_Package_LPA and Avg_Package_LPA) all correspond to the unmatched-record counts identified in Section 5, confirming that no new or unexplained missing data was introduced by the merge process itself.

8. Summary of Results

Metric	Value
Total students in the final dataset	1,000
Students with no academic record	40
Students with no skills-assessment record	130
Students who never interviewed	570
Students placed (at least one offer)	362
Placement rate among students who interviewed	84.2%
Students with more than one interview	180
Average CGPA (students with a record)	7.17
Average package among placed students	6.39 LPA

9. Conclusion

The integration was completed using a left join from Student Details, driven directly by the requirement that every registered student remain in the final dataset, combined with a pre-merge aggregation of the one-to-many Placement Information table. The resulting file, Final_Integrated_Dataset.csv, contains exactly 1,000 rows with no duplicate identifiers and no duplicate records. Every remaining missing value in the dataset corresponds to a genuinely absent source record rather than an integration error, and at no point in the process was a student

record or an interview record deleted without justification. All cleaning and integration decisions, together with the validation evidence supporting them, are documented in the accompanying notebook and summarised with screenshots in this report.